Ejemplo sobre punteros

Arquitectura del Computador - LCC - FCEIA-UNR

29 de julio de 2026

Dado el siguiente código en lenguaje C `punteros.c`:

```c
#include <stdio.h>
int main(){
    int a=145;        //Declaración de variable entera de tipo entero
    int *puntero;     //Declaración de variable puntero de tipo entero
    puntero = &a;     //Asignación de la dirección memoria de a

    printf("El valor de a es: %d\n", a);
    printf("El valor de *puntero es: %d\n" , *puntero);
    printf("La dirección de memoria de *puntero es: %p\n", puntero);

    return 0;
}
```

Si compilamos y ejecutamos el código, la salida es la siguiente:

```
$ ./a.out
El valor de a es: 145
El valor de *puntero es: 145
La dirección de memoria de *puntero es: 0x7ffc33bd4954
```

Ahora compilemos utilizando la opción `-g` para poder *debuggear* utilizando GDB:

```
$ gcc -g punteros.c
```

y ejecutemos con GDB:

```
$ gdb ./a.out
```

Una vez que estamos dentro de la sesión de *debugging*, ponemos un *breakpoint* en `main` y ejecutamos el comando `run`. Luego vamos ejecutando línea a línea hasta ejecutar los dos `printf`:

```
(gdb) br main
Breakpoint 1 at 0x40050e: file punteros.c, line 5.
(gdb) r
Starting program: /home/dferoldi/2020/a.out

Breakpoint 1, main () at punteros.c:5
5 int a=145; //Declaración de variable entera de tipo entero
(gdb) n
7 puntero = &a; //Asignación de la dirección memoria de a
(gdb) n
9 printf("El valor de a es: %d\nEl valor de *puntero es: %d\n",a,*puntero);
(gdb) n
El valor de a es: 145
El valor de *puntero es: 145
```

```
10 printf("La dirección de memoria de *puntero es: %p\n",puntero);
(gdb) n
La dirección de memoria de *puntero es: 0x7fffffffe1d4
12 return 0;
```

Ahora verifiquemos lo anterior utilizando comandos de GDB. En primer lugar, podemos usar el comando `print` para ver el contenido de la variable `a`, luego podemos imprimir a puntero dicha variable y finalmente examinar el contenido de la memoria utilizando el comando `examine`[1]:

```
(gdb) p a
$1 = 145
(gdb) p &a
$2 = (int *) 0x7fffffffe1d4
(gdb) x/d &a
0x7fffffffe1d4: 145
```

Efectivamente, vemos que en la dirección `0x7fffffffe1d4`, que es la dirección almacenada en la variable tipo puntero a entero `puntero`, se encuentra almacenado el valor `145`.

A continuación vemos una posible implantación en Assembler x86-64 del código anterior:

```
.data
.LC0: .string "El valor de a es: %d\n"            # Cadena para imprimir el valor de a
.LC1: .string "El valor de *puntero es: %d\n"     # Cadena para imprimir el valor apuntado
.LC2: .string "La dirección de memoria de *puntero es: %p\n" # Cadena para imprimir la dirección

.text
.global main
main:
    pushq   %rbp                # Guarda el base pointer anterior
    movq    %rsp, %rbp          # Establece el nuevo marco de pila
    subq    $16, %rsp           # Reserva 16 bytes en la pila (variables locales)

    movl    $145, -12(%rbp)     # a = 145 (variable local en la pila)
    leaq    -12(%rbp), %rax     # Carga la dirección de a en %rax
    movq    %rax, -8(%rbp)      # puntero = &a (almacena la dirección en otra variable local)

    movl    -12(%rbp), %eax     # Carga el valor de a en %eax
    movl    %eax, %esi          # Segundo argumento de printf (valor de a)
    movl    $.LC0, %edi         # Primer argumento: formato
    movl    $0, %eax            # Convención ABI: limpiar %eax para llamadas variádicas
    call    printf              # printf("El valor de a es: %d\n", a)

    movq    -8(%rbp), %rax      # Carga el puntero (&a)
    movl    (%rax), %eax        # Dereferencia: obtiene el valor apuntado (*puntero)
    movl    %eax, %esi          # Segundo argumento: *puntero
    movl    $.LC1, %edi         # Primer argumento: formato
    movl    $0, %eax
    call    printf              # printf("El valor de *puntero es: %d\n", *puntero)

    movq    -8(%rbp), %rax      # Carga nuevamente el puntero
    movq    %rax, %rsi          # Segundo argumento: dirección almacenada en puntero
    movl    $.LC2, %edi         # Primer argumento: formato
    movl    $0, %eax
    call    printf              # printf("La dirección de memoria de *puntero es: %p\n", puntero)

    movl    $0, %eax            # Valor de retorno de main = 0
    leave                       # Restaura el marco de pila (equivale a mov %rbp, %rsp; pop %rbp)
    ret                         # Retorna al sistema
```

[1] Utilizamos la opción `d` para ver el resultado en formato decimal.